



Universidad
Indoamérica

UNIT 8 · FUNDAMENTALS OF OPERATING SYSTEMS

Networking: sockets

Universidad Tecnológica Indoamérica

Lab TD7

Two connection modes

Connected · `SOCK_STREAM` (TCP)

- The connection is set up once; then data is exchanged
- Like a **telephone**
- A reliable **byte stream**, like a pipe: detects errors, retransmits, keeps the order

Connectionless · `SOCK_DGRAM` (UDP)

- Each send is an independent message (**datagram**)
- Like a **telegram**
- No guarantees: loss, duplication, reordering

Stream versus datagrams, measured

Real output

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom #3: 5
"three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

- TCP does not keep boundaries: **the application protocol** marks where each message ends (a line, a length...)
- UDP loses datagrams **without telling anybody**, even on the same machine

```
int socket(int domain, int type, int protocol);
```

domain	AF_UNIX : local, the address is a path · AF_INET : IPv4 · AF_INET6 : IPv6
type	SOCK_STREAM : connected · SOCK_DGRAM : connectionless · SOCK_RAW : raw packets (needs CAP_NET_RAW)
+ flags	SOCK_NONBLOCK , SOCK_CLOEXEC (Linux, since 2.6.27): combined with in type
protocol	0 : the system chooses · IPPROTO_TCP · IPPROTO_UDP

Fix (slide 182): "AF_UNIX: local (127.0.0.1)" mixes two things. 127.0.0.1 is AF_INET over the loopback interface; AF_UNIX does not use IP.

Local domain: a file of type socket

Real output

```
$ ./local fso.sock  
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108 bytes  
the server asks the kernel who connected: pid = mine, uid 1000 (my user)  
read 5 bytes; after close the file stays until unlink  
still there after close: yes
```

- `bind` creates an entry in the file system (the `s` of `ls -l`); it stays until `unlink`
- The kernel tells the server **who** connected (`SO_PEERCRED`): that is how `systemd`, `D-Bus` or `/var/run/docker.sock` decide permissions

Internet domain: address + port

- A port (16 bits) identifies the service within the machine
- Ports < 1024: privileged only
(`CAP_NET_BIND_SERVICE`); in Linux the limit is `net.ipv4.ip_unprivileged_port_start`
- Service names: `/etc/services`

Real output

```
$ ./addr 7000
...
ip_unprivileged_port_start = 1024
sin_port = htons(80):      bind: Permission denied
...
```

SERVICE	PORT	TODAY
ftp	21/tcp	unencrypted; sftp or https is used
telnet	23/tcp	unencrypted; replaced by ssh (22/tcp)
http	80/tcp	almost always redirects to https
https	443/tcp, 443/udp	TLS; HTTP/3 runs over UDP (QUIC)

bind : address + port

C

```
int bind(int sock, const struct sockaddr *addr, socklen_t len)
{
    struct sockaddr { sa_family_t sa_family; char sa_data[14]; };
    struct sockaddr_un { /* AF_UNIX (= AF_LOCAL) */
        sa_family_t sun_family;
        char sun_path[108]; /* a path */
    };
    struct sockaddr_in { /* AF_INET */
        sa_family_t sin_family;
        in_port_t sin_port; /* network order */
        struct in_addr sin_addr; /* 4 bytes */
        unsigned char sin_zero[8];
    };
    /* AF_INET6: struct sockaddr_in6 (16-byte address) */
```

- Each domain has its own structure; all are passed as `struct sockaddr *` with their size
- `struct sockaddr_storage` can hold any of them (for example, in `accept`)

Real output

```
$ ./addr 7000
...
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28,
sockaddr_un 110, sockaddr_storage 128
```

Slide 187: htons was missing

C

```
/* slide 187 */  
s.sin_port = 7000;           /* wrong */  
/* fixed (sock.c) */  
s.sin_port = htons(7000);    /* network order */  
s.sin_addr.s_addr = htonl(INADDR_ANY);
```

- The network uses *big-endian* order; x86-64 and ARM64 are *little-endian* (you saw it in TD1 with `od`)

0x1B58 read the other way round is 0x581B = **22555**: the server listens on another port and nobody finds it.

Real output

```
$ ./addr 7000  
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order  
sin_port = 7000:           the kernel bound port 22555  
sin_port = htons(7000):    the kernel bound port 7000  
...
```

From `gethostbyname` to `getaddrinfo`

C tds/td7/addr.c

```
const char *names[][2] = { { "localhost", "http" }, { "
for (int k = 0; k < 2; k++) {
    struct addrinfo hints = { .ai_family = AF_UNSPEC, .
    int e = getaddrinfo(names[k][0], names[k][1], &hints,
    exit_if(e != 0, gai_strerror(e));
    for (struct addrinfo *p = res; p != NULL; p = p->ai
...
    freeaddrinfo(res);
}
```

Real output

```
$ ./addr 7000
...
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
...
```

- `gethostbyname` (slide 187): IPv4 only, not thread-safe, out of POSIX since 2008
- `getaddrinfo` (RFC 3493): name + service → list of IPv4 and IPv6 addresses, ready for connect

`int listen(int sock, int backlog);`

- Declares the service to the system: the socket becomes **passive**, it only waits for connections
- `backlog`: how many **already established** connections can wait for `accept` (Linux admits backlog + 1)
- With the queue full, the SYN is dropped and the client resends it (1 s, 2 s, 4 s...)

Real output

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

accept : taking a connection from the queue

C

```
int accept(int sock, struct sockaddr *a, socklen_t *len);  /* a new descriptor */
```

- `a` and `*len` are filled with the client's address (or `NULL` if not needed)
- Blocks until there is a connection in the queue
- Returns a **new descriptor** for that client; the listening one keeps accepting

C tds/td7/sock.c

```
/* 5. accept blocks until a client connects and returns  
int client = accept(s, NULL, NULL);  
exit_if(client == -1, "accept");  
printf("connection accepted, closing\n");
```

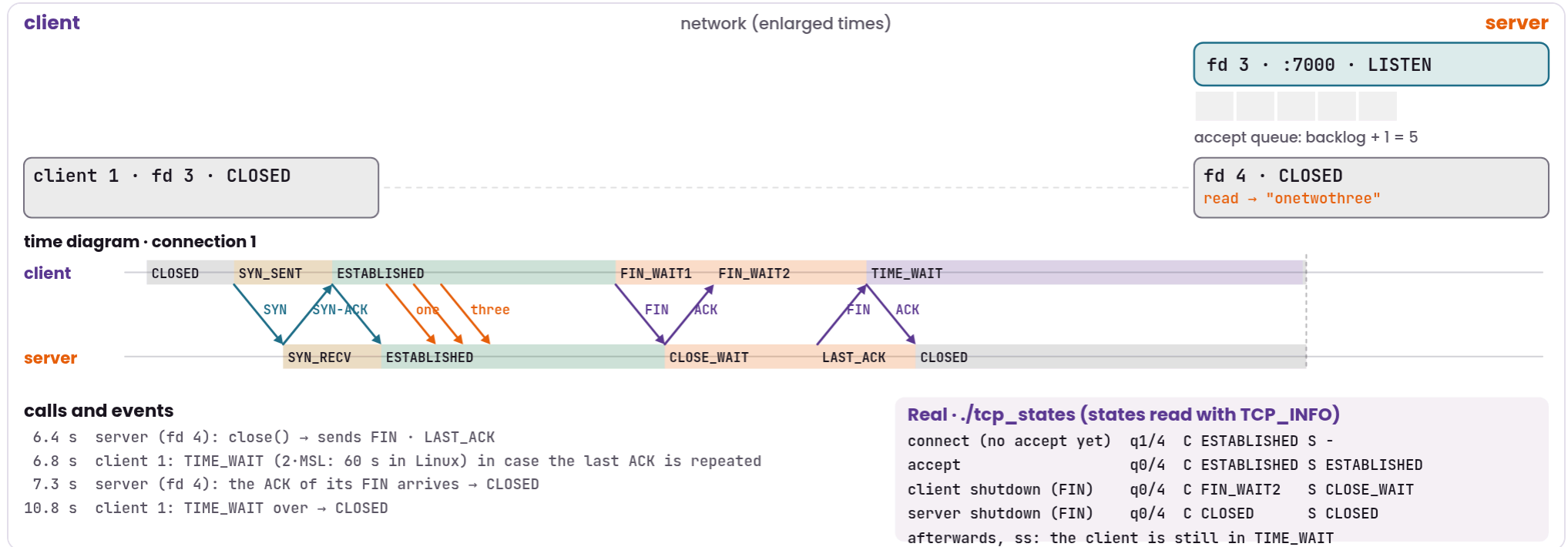
Fix (slide 189): it returns a descriptor ≥ 0 , not " > 0 " (0 is valid if standard input was closed), or -1 on error. `accept4(..., SOCK_CLOEXEC)` keeps the descriptor from leaking into an `exec`.

`connect` : requesting the connection

C

- Requests the connection to the server's address: the kernel does the **three-way handshake** (SYN, SYN-ACK, ACK)
- Blocks until the handshake ends: **it does not wait for** `accept`
- Typical errors: `ECONNREFUSED` (nobody listens on that port), `ETIMEDOUT`
- With `SOCK_NONBLOCK` it returns `EINPROGRESS` and the handshake goes on by itself (that is how `backlog.c` opens 8 connections at once)

A connection, segment by segment



With the descriptor returned by `accept`

- Write: `write(fd, buf, n)` or `send(fd, buf, n, flags)`
- Read: `read(fd, buf, n)` or `recv(fd, buf, n, flags)`
- Close: `close(fd)`; `shutdown(fd, SHUT_WR)`
closes only the writing direction (sends FIN)

What you know from units 2 and 5 applies here:

- **Partial** reads and writes: loop (`write_all`)
- `read` → 0: the other end closed
- Writing to a closed socket → `SIGPIPE`; avoid it with `MSG_NOSIGNAL` or by ignoring the signal

The states, read from the kernel with TCP_INFO

Real output

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -           | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Connectionless mode: `sendto` and `recvfrom`

C

```
ssize_t sendto(int sock, const void *buf, size_t len, int flags,  
               const struct sockaddr *dest, socklen_t dlen);  
ssize_t recvfrom(int sock, void *buf, size_t len, int flags,  
                 struct sockaddr *src, socklen_t *slen);
```

- `sendto` = `connect` + `send` in one call; `recvfrom` = `accept` + `recv`, and says who sent
- They return the bytes sent or received (-1 on error); one `recvfrom` = **one whole datagram** (if the buffer is short, the rest is lost)
- You can also `connect` a UDP socket: it fixes the destination and allows `send` / `recv`

Who calls what, in order

CONNECTED (TCP)	
SERVER	CLIENT
socket · bind · listen	socket
accept (blocks)	connect → handshake
read ⇔ write	write ⇔ read
close	close

CONNECTIONLESS (UDP)	
SERVER	CLIENT
socket · bind	socket
recvfrom (blocks)	sendto
sendto	recvfrom
close	close

Only the server binds a well-known port; the client automatically gets an ephemeral port.

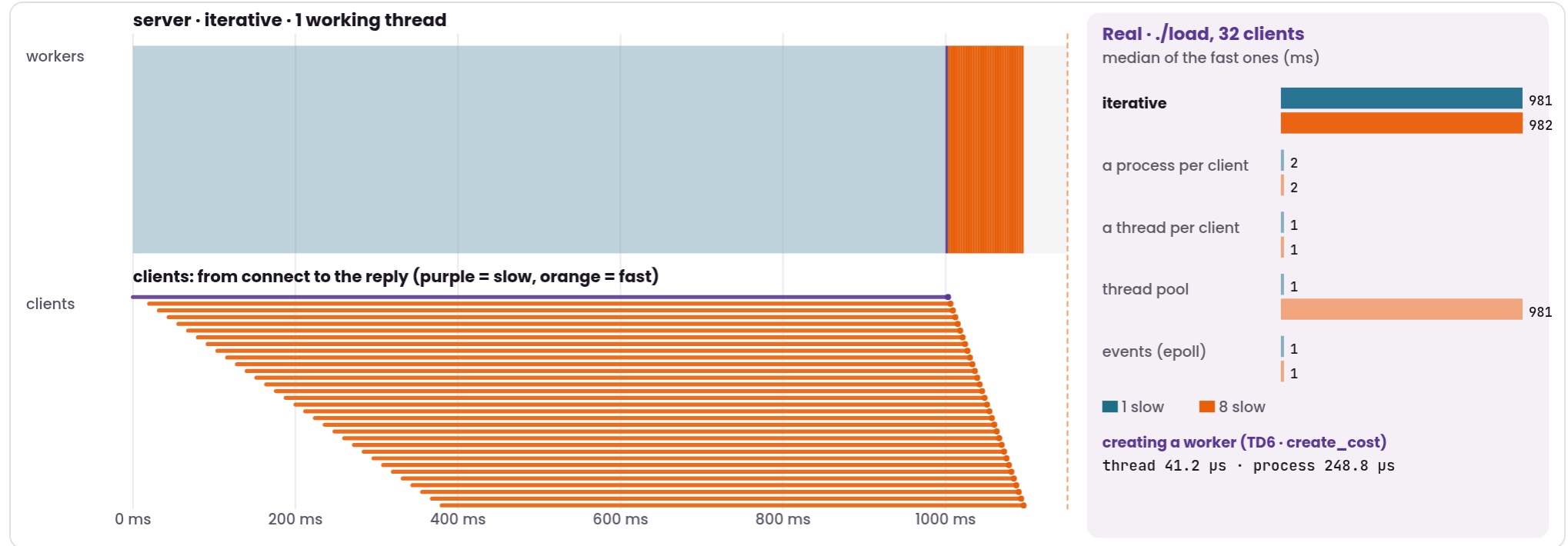
Server models

- **Iterative:** one client at a time
- **Multiprocess or multithreaded:** the main thread does `accept`, creates a process or thread for the service and goes back to `accept`
- **Thread pool:** the same, with a limited number of threads. Producer: the main thread (connections); consumers: the service threads

Today also

Event loop: one thread and `epoll` (Linux, 2002) serve thousands of connections; it is the answer to the C10K problem (D. Kegel, 1999) and the model of nginx or Node.js.
`io_uring` (Linux 5.1, 2019) takes the idea further: queues shared with the kernel.

Who waits for whom?



32 clients; first 1, then 8 slow ones (1 s)

Real output

```
$ ./http_server -m iterative & ./load PORT 1 31 1000;  
./load PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 981 ms, max 982 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 982 ms, max 983 ms
```

Real output

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load  
PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 1 ms, max 2 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 1 ms, max 2 ms
```

The iterative server makes everybody wait for a single slow client; `epoll`, with a single thread, does not. The pool of 8 copes with 1 slow client but not with 8 (A8.2).

Conclusion

- A socket is a descriptor: `read`, `write` and `close` as with a file or a pipe
- TCP: reliable stream, connection made by the kernel; UDP: datagrams without guarantees
- Addresses in network order (`htons`) and resolved with `getaddrinfo`
- The server model decides who waits: processes, threads, pool or events

Solved TD7 + AI challenge

- `sock`, `echo_server` (iterative and with `fork`)
- `http_server` with five models
- New: `tcp_states`, `backlog`, `load`, `udp`, `addr`, `local`

Bash

```
make -C tds && tds/tests/run.sh td7
```



Universidad
Indoamérica

TAKE-AWAYS

Three questions

1. Why can `connect` finish before the server calls `accept`?
2. Which port does `sin_port = 7000` open on your laptop, and why?
3. Which server model copes with 8 slow clients using a single thread?